破除AI Agent自主操控风险：万字解读LangGraph“人工干预”机制，附零基础实战

原创 腾讯程序员 腾讯技术工程 2025年12月15日 02:18 广东 258人



作者：yukixxwang

> 随着大型语言模型（LLM）驱动的自主代理（Agent）从学术走向应用，如何确保其行为的可靠性、安全性与可控性，已成为决定其能否在真实世界关键任务中落地的核心挑战。大语言模型能力虽然越来越强，但并非完美无缺，可能产生错误或不准确输出。当一个 Agent 被授权执行高风险领域或敏感操作时，一个小小的错误也可能带来不可预知的风险。所以我们需要“人工干预”机制，在关键决策点让 Agent 停下来，将控制权交还给人类，让人类的智慧能够介入，弥补模型的不足。

引言

随着大型语言模型（LLM）驱动的自主代理（Agent）从学术走向应用，如何确保其行为的可靠性、安全性与可控性，已成为决定其能否在真实世界关键任务中落地的核心挑战。大语言模型能力虽然越来越强，但并非完美无缺，可能产生错误或不准确输出。当一个 Agent 被授权执行高风险领域或敏感操作时，一个小小的错误也可能带来不可预知的风险。所以我们需要“人工干预”机制，在关键决策点让 Agent 停下来，将控制权交还给人类，让人类的智慧能够介入，弥补模型的不足。

全文概览

在介绍Multi-agent的人工干预机制之前，我们先简单介绍Multi-Agent的基本概念：定义、主流开发框架、人工干预。之后我们会着重以LangGraph为例，介绍LangGraph的人工干预机制的核心原理、四大经典模式、 以及具体的案例实践。在案例实践中，我们除了介绍LangGraph的四大经典模式案例之外，还通过MCP协议提供的智能搜索工具（Venus MCP server市场 - 网络综合搜索工具）搭建了真实案例来帮助大家理解LangGraph的中断机制。

什么是Multi-Agent?

简单来说，Multi-Agent（多智能体）系统不是由一个“无所不知”的超级AI来解决所有问题，而是由多个具有特定角色和能力的、相对简单的自主智能体（Agent）协同工作，共同完成一个

复杂任务的系统。这些智能体各自有自己的专长（通过不同的Prompt、工具和知识库来定义），它们之间可以沟通、协作、互相反馈、甚至辩论，最终合力交付一个高质量的成果。

核心特征：

- 分解 (Decomposition): 将一个宏大、模糊的任务分解成多个具体、可执行的子任务。
- 专长 (Specialization): 每个智能体都有明确的角色和擅长的技能（例如，一个智能体专门用于网络搜索，另一个专门用于代码执行）。
- 协作 (Collaboration): 智能体之间通过信息交换（类似内部聊天）来协调工作。例如，程序员写完代码后交给测试员。
- 自主性 (Autonomy): 每个智能体可以在其职责范围内独立做出判断和执行操作，无需人类每一步都进行干预。

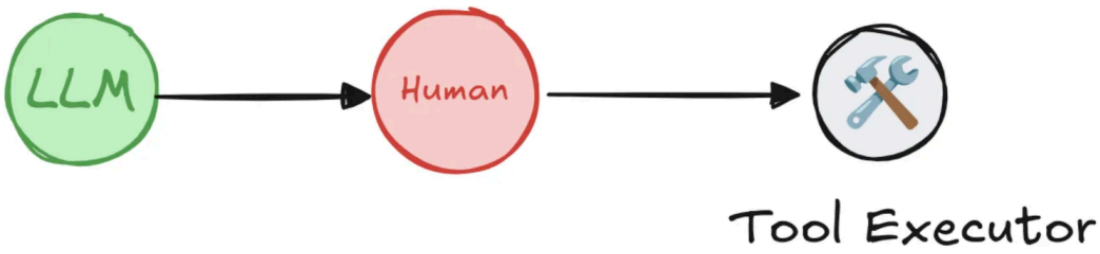
Multi-Agent主流开发框架

随着LLM（大型语言模型）的发展，多智能体框架也迎来了爆发式增长。它们封装了智能体定义、通信、任务调度等复杂逻辑，让开发者能更专注于业务逻辑。目前主流的开发框架主要有：LangGraph、AutoGen、CrewAI、MetaGPT和Magentic。

Multi-Agent中的人工干预

什么是人工干预

简单来说，就是让人类能够参与到机器的工作流程中，深入到AI的核心工作环节，让人类能够实时审查、编辑甚至批准AI的决策和行动。尤其是在由大语言模型驱动的应用场景中，这种机制显得尤为重要。因为LLM虽然很强大，但有时候也会犯错，或者需要一些额外的背景知识才能做出正确的判断。这时候就需要人类来帮忙把关。



引入人工干预的必要性

大语言模型能力越来越强，写文章、写代码、翻译、做数学等，但并非完美无缺，可能产生错误或不准确输出。当一个 Agent 被授权执行预订酒店、调用付费 API 、修改数据库、或遇到法律、医疗这种高风险领域等敏感操作时，在拥有完全的自主性的情况下，甚至一个小小的错误也可能带来不可预知的风险。

所以我们需要一种机制，需要一个“暂停按钮”，在关键决策点让 Agent 停下来，将控制权交还给人类，让人类的智慧能够介入，结合人类的判断力、专业知识和经验给AI的工作加一道保险，弥补模型的不足。人类还可以通过审核、修正、验证等操作，提高应用的准确性和可靠性。在处理复杂或敏感任务时，人工干预能够提供更可靠的保障。

LangGraph中的人工干预

LangGraph 框架通过其强大的“人机协同”（Human-in-the-Loop）功能，提供了一套优雅而完备的解决方案。本文接下来将深入剖析 LangGraph 如何通过持久化状态与动态中断机制，实现灵活、可靠的人工干预，并详解其在实践中的四大核心设计模式。

LangGraph 框架通过其创新的 **interrupt（中断）**机制，使得构建需要人工审查、编辑和批准的“人机协同”（Human-in-the-Loop）工作流成为可能。当工作流（Graph）执行到中断点时，它会保存当前的所有状态，然后无限期暂停，直到接收到人类的输入指令后再从断点处继续。这为构建可靠、安全且透明的 Agent 应用奠定了基石。

它允许用户在工作流的任何阶段进行干预。这对于大型语言模型驱动的应用程序尤其有用，因为模型输出可能需要验证、更正或补充上下文。该功能包括两种中断类型：**动态中断**和**静态中断**，允许用户暂停图执行并进行审查或编辑。此外，灵活的集成点使人类可以针对特定步骤进行干预，例如批准 API 调用、更正输出或引导对话。

作用及应用场景

- 对关键步骤（如外部 API 调用）进行人工批准/拒绝，防止错误执行。
- 纠正或补充模型输出，提升结果可靠性。
- 让业务人员在不阻塞整个系统的情况下提供上下文或修正。

核心能力

LangGraph 的人机协同能力构建于两大基石之上：持久化的执行状态 和 灵活的中断机制。

持久化执行状态

这是实现异步、无时间限制人工审查的关键。LangGraph 在工作流（Graph）的每一步执行后，都会利用其**持久化层（Persistence Layer）**来创建检查点（Checkpoint），完整地保存当前 Graph 的所有状态。

这意味着，当一个工作流被中断时，它的全部上下文都被安全地保存下来。人类可以在任何时候（几秒、几小时甚至几天后）回来处理这个中断，然后系统可以从中断点无缝恢复，继续执行后续任务，而不会丢失任何信息。

灵活的中断机制

- 动态中断 (Dynamic Interrupts)

在特定节点内部根据当前状态暂停，这种方式就像在程序里设置了一个条件判断，当满足某个特定条件时，就自动触发中断。例如，当你的LLM在生成一段文本后，你希望检查一下这段文本是否符合某些特定的要求（是否有敏感词、信息是否准确等），如果发现不符合要求，就可以动态触发中断，把工作流暂停下来，等待人工介入。这种方式非常灵活，可以根据实际情况随时调整中断的条件，真正做到按需暂停。

- 静态中断 (Static Interrupts)

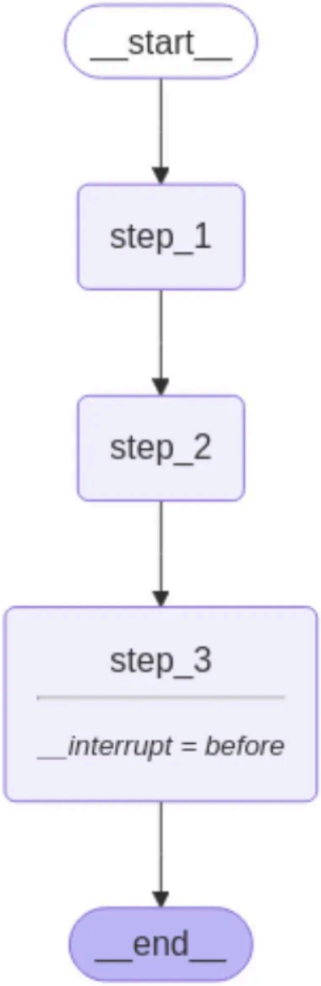
在预定义的节点前后固定设置。中断后图会暂停，状态持久化，等待人工操作后再 resume。

这种方式比较直接，它是在工作流设计阶段预先定义好的。你可以指定在某个特定的节点之前或者之后，必须暂停工作流，等待人工干预。或者，在某个关键步骤之后，需要人工确认结果是否正确，才能继续下一步。静态中断就像是在工作流中设置了几个固定的“关卡”，每到一个关卡，就必须有人来检查一下，确保万无一失。

触发方式： 使用 **interrupt_before** 和 **interrupt_after**，在预定义的节点前后暂停。

适用场景：需要在固定流程节点进行人工审核或确认的场景。

示例：在 API 调用前使用 interrupt_before，确保 API 请求的合规性。



An example graph consisting of 3 sequential steps with a breakpoint before step_3.

灵活的集成点

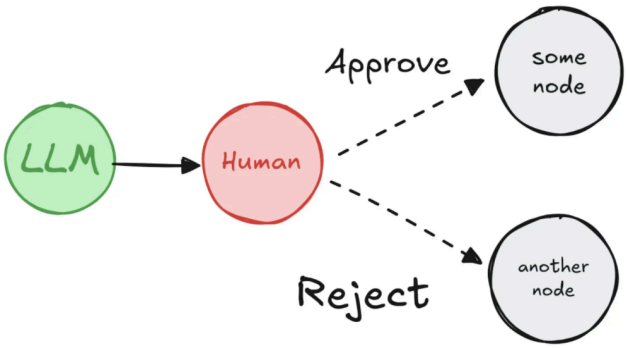
LangGraph的人工干预机制还有一个非常重要的特点，就是它的集成点非常灵活。也就是说，你可以把人工干预的逻辑放在工作流的任何位置。你可以根据不同的需求，选择在不同的节点进行人工干预。比如，你想让人类审批API调用，那就把中断点放在API调用节点之前；你想让人类纠正LLM的输出，那就把中断点放在LLM生成输出之后。这种灵活性使得我们可以根据具体的业务场景，定制化地设计人工干预的流程，真正做到精准定位，避免不必要的干预。

典型模式

基于上述强大的功能，我们可以构建出各种各样的典型应用场景。

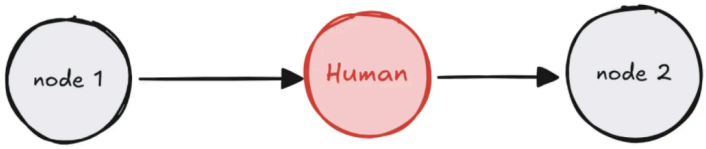
- **模式一：批准/拒绝**

在工作流的关键步骤前暂停一下，让人类来审核一下，看看这个操作是不是应该执行。如果审核通过，就继续执行；如果审核不通过，就可以拒绝这个操作，甚至可以采取一些替代方案。



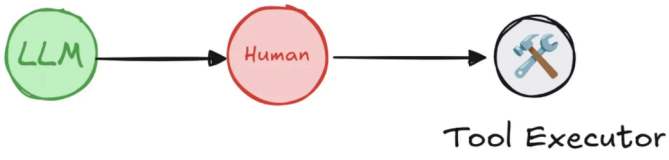
- **应用场景**：API调用前的审批、敏感操作确认（财务交易确认、订单确认）
- **价值**：降低风险，防止错误操作，提高安全性。
- **模式二：编辑图状态**

暂停后让人工修改状态后再继续。有时候，LLM在生成结果的过程中，可能会出现一些错误，或者信息不够完整。这时候，我们就可以暂停工作流，让人类来审核当前的状态，并进行修改。修改完成后，再把更新后的状态重新放回工作流中，让后续的步骤继续执行。这样就能保证整个工作流的数据质量，避免因为错误的信息而导致后续步骤出现问题。



- 应用场景：纠正错误信息（纠正用户姓名拼写错误）、补充缺失信息、更新上下文等。
- 价值：修正错误，完善信息，提升后续步骤的准确性。
- **模式三：审查工具调用**

在 LLM 发出工具请求前让人工检查并可编辑。我们知道，LLM往往需要借助各种工具来完成任务，比如搜索网络信息、查询数据库等。但是，LLM有时候可能会选错工具，或者调用工具的参数设置不正确。为了避免这种情况，我们可以再LLM发起工具调用之前，先暂停工作流，让人类来审核一下这个工具调用是否合理。比如，LLM可能想调用一个支付API来完成转账，但是参数设置错了，导致金额输入错误。这时候，人类就可以介入，检查一下工具调用的参数是否有错误，若有错误就及时纠正。



- 应用场景：审核API请求参数、验证工具选择的合理性等。

- 价值：确保工具调用的正确性和安全性，避免错误操作。

- **模式四：验证人工输入**

在后续步骤前确认人工提供的信息有效。这个模式听起来好像有点反直觉，因为我一直在讲“human-in-the-loop”，怎么又变成验证人类输入了呢？其实，这个模式主要是针对那些需要用户输入信息的场景。比如，用户填写一个表单，或者在聊天机器人中输入一些指令。为了确保用户输入的信息是有效的，我们可以利用人工干预机制，在系统处理用户输入之前，先暂停一下，让用户自己确认一下输入的信息是否正确。如果用户确认无误，就继续执行；如果用户发现输入有误，可以及时修改。



- **应用场景**：用户输入验证、表单数据校验等。

- **价值**：确保数据质量，防止无效或错误的输入影响后续流程。

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号

langGraph中的人工干预核心工作流

实现一次完整的人机交互闭环，interrupt 的工作流遵循一个清晰的四步模式：

1. **配置持久化层 (Checkpointer)**：中断的本质是状态的保存与恢复。因此，在编译 Graph 时，必须为其指定一个 checkpointer，用于在每一步执行后自动保存状态。

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = graph_builder.compile(checkpointer=checkpointer)
```

2. **在节点中调用 interrupt()**：在需要人工干预的节点函数中，调用 interrupt() 函数。此函数会立即暂停执行，并可以向用户传递一个 JSON 可序列化的对象，其中包含需要审查的数据。

```
from langgraph.types import interrupt
```

```python
def human_review_node(state: State):
    # 中断执行，并将 state 中的摘要文本交给用户审查
    edited_data = interrupt({
        "task": "请审查并编辑下面的摘要",
        "summary_to_review": state["summary"]
    })
    # 恢复后，edited_data 将是用户输入的新内容
    return {"summary": edited_data["edited_summary"]}
```

3. **运行并触发中断**：使用 invoke 或 stream 方法并传入唯一的 thread_id 来运行 Graph。当执行流遇到 interrupt() 时，Graph 会暂停，并在返回结果中包含一个特殊的 **interrupt** 键，其中包含了中断的详细信息（如传递给用户的数据）。

```python
config = {"configurable": {"thread_id": "some-unique-id"}}
result = graph.invoke({"summary": "初步生成的摘要..."}, config=config)

# 检查中断信息
print(result['__interrupt__'])
# > [Interrupt(value={'task': '请审查...', 'summary_to_review': '...'}, id='...')]
```

4. **使用 Command 恢复执行**：当用户完成审查并提供输入后，通过再次调用 invoke 或 stream，并传入一个 Command(resume=...) 对象来恢复 Graph 的执行。resume 中包含的值将作为 interrupt() 函数的返回值。

```python
from langgraph.types import Command

# 用户提供了编辑后的摘要
user_input = {"edited_summary": "这是经过人工编辑的最终摘要。"}
final_result = graph.invoke(Command(resume=user_input), config=config)
```

⚠️ **核心机制警示：恢复即重跑** (Resume Reruns the Node)

这是理解 interrupt 最关键的一点：恢复执行并非从 interrupt() 函数调用的那一行代码继续，而是从包含 interrupt() 的那个节点的开头重新执行整个节点。

在重跑期间，当执行流再次遇到 interrupt() 时，它不会再次暂停，而是直接返回 Command(resume=...) 中提供的值。这个设计虽然巧妙，但也意味着任何位于 interrupt() 调用之前的、具有副作用的操作（如 API 调用、数据库写入）都会被重复执行。因此，最佳实践是将副作用操作放在 interrupt() 之后，或置于一个独立的后续节点中。

langGraph实战模式--interrupt的四大经典模式的应用

基于其核心机制，interrupt 可以灵活地实现多种强大的人机交互模式。

要在图中使用interrupt，您需要：

1. 指定一个检查点来保存每个步骤后的图形状态。
2. interrupt()在适当的地方调用。
3. 使用线程ID运行图，直到interrupt命中。
4. 使用invoke/恢复执行stream。

我们挑选了两个模式（模式一和模式三），边运行边讲解执行过程。

模式一：审批或否决 (Approve or Reject)

在执行高风险操作前，强制要求人工批准。根据用户的决策，Graph 可以走向不同的分支。

步骤一：基本函数定义

```python
# 目标：中断图的执行，让用户做出决策（如批准/拒绝），然后根据决策跳转到不同的流程分支。

from typing import Literal, TypedDict
import uuid

from langgraph.constants import END
from langgraph.graph import StateGraph
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver

# 1. 定义图的共享状态，包含一个 'decision' 字段来记录人类的决定
class State(TypedDict):
    llm_output: str
    decision: str

# 模拟一个生成内容的节点
def generate_llm_output(state: State) -> State:
    print("\n--- 步骤1：AI生成内容 ---")
    return {"llm_output": "这是AI生成的一段需要审批的文本。"}

# 2. 定义人工审批节点。注意：返回值类型是 Command，意味着此节点将发出控制指令。
def human_approval(state: State) -> Command[Literal["approved_path", "rejected_path"]]:
    """
    此节点暂停并等待人类决策，然后根据决策返回一个带有 `goto` 指令的 Command，
    从而控制图的走向。
    """
    print("\n--- 暂停：等待人工审批 ---")
    # 3. 暂停图的执行，等待人类做出决策（例如，输入 "approve" 或 "reject"）。
    decision = interrupt({
        "question": "请审批以下内容，回复 'approve' 或 'reject'：",
        "llm_output": state["llm_output"]
```

```
    })

    # 4. 核心逻辑：根据人类的决策（'decision' 变量的值）进行判断。
    if decision == "approve":
        print("\n--- 决策：批准 ---")
        # 5. 如果批准，返回一个 Command 指令，强制图跳转到 'approved_path' 节点。
        #    'goto' 是实现条件路由的关键。'update' 是一个可选参数，用于同时更新状态。
        return Command(goto="approved_path", update={"decision": "approved"})
    else:
        print("\n--- 决策：拒绝 ---")
        # 6. 如果拒绝，则跳转到 'rejected_path' 节点。
        return Command(goto="rejected_path", update={"decision": "rejected"})

# 批准后的流程节点
def approved_node(state: State) -> State:
    print("--- 步骤2 (分支A)：已进入批准流程。---")
    return state

# 拒绝后的流程节点
def rejected_node(state: State) -> State:
    print("--- 步骤2 (分支B)：已进入拒绝流程。---")
    return state
```

步骤二：构建图

```
builder = StateGraph(State)
builder.add_node("generate_llm_output", generate_llm_output)
builder.add_node("human_approval", human_approval)
builder.add_node("approved_path", approved_node)
builder.add_node("rejected_path", rejected_node)

# 7. 设置图的入口和边，定义了基本的流程。
#    注意，从 human_approval 节点出发的路径将由其返回的 Command(goto=...) 动态决定。
builder.set_entry_point("generate_llm_output")
builder.add_edge("generate_llm_output", "human_approval")
builder.add_edge("approved_path", END) # 批准分支的终点
builder.add_edge("rejected_path", END) # 拒绝分支的终点

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)
```

步骤三：首次运行，图会执行到 human_approval 节点并暂停

```
config = {"configurable": {"thread_id": f"thread-{uuid.uuid4()}"}}
print("首次调用，启动审批流程...")
```

```
result = graph.invoke({}, config=config)
print("\n图已暂停，等待审批...")
```

运行结果：

```
首次调用，启动审批流程...
--- 步骤1：AI生成内容 ---
--- 暂停：等待人工审批 ---
图已暂停，等待审批...
```

步骤四：使用 Command(resume="approve") 恢复执行：

```
# "approve" 这个字符串会成为 interrupt() 的返回值，并赋给 'decision' 变量。
#   你可以尝试将 "approve" 改为 "reject" 来测试另一条分支。
print("\n--- 恢复执行：传入 'approve' 决策 ---")
final_result = graph.invoke(Command(resume="approve"), config=config)

# 打印最终结果。由于我们 resume="approve"，流程会走 approved_path，最终状态会包含 'decision': 'a
print("\n流程执行完毕，最终状态如下：")
print(final_result)
```

运行结果：

```
--- 恢复执行：传入 'approve' 决策 ---
--- 暂停：等待人工审批 ---
--- 决策：批准 ---
--- 步骤2（分支A）：已进入批准流程。---
流程执行完毕，最终状态如下：
{'llm_output': '这是AI生成的一段需要审批的文本。', 'decision': 'approved'}
```

模式二：审查与编辑状态 (Review and Edit State)

允许用户审查并直接修改 Agent 在执行过程中生成的数据，如修正一篇 AI 生成的文章草稿。

```
# 模式二：编辑状态 (Edit State)
# 目标：中断图的执行，让用户提供或修改数据，然后用该数据更新图的状态。

# 导入所需库
from typing import TypedDict
import uuid

from langgraph.constants import START, END
from langgraph.graph import StateGraph
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver

# 1. 定义图的状态 (State)
```

```python
# State 是一个 TypedDict，它定义了在整个图的执行过程中共享的数据结构。
# 把它想象成所有节点都可以读写的共享内存。
class State(TypedDict):
    summary: str

# 2. 定义图中的各个节点 (Node)
# 每个节点都是一个函数，它接收当前状态作为输入，并返回一个字典来更新状态。

# 节点A：模拟AI生成摘要
def generate_summary(state: State) -> State:
    print("--- 步骤1: AI 正在生成摘要... ---")
    return {
        "summary": "The cat sat on the mat and looked at the stars."
    }

# 节点B：人工审查和编辑节点
def human_review_edit(state: State) -> State:
    print("--- 步骤2: 等待人工审查和编辑... ---")
    # 调用 interrupt() 来暂停图的执行。这是实现人机交互的关键。
    # 传入的字典会作为中断的有效负载（payload）发送给调用方（例如前端应用），
    # 以便向用户展示任务和当前数据。
    result = interrupt({
        "task": "Please review and edit the generated summary if necessary.",
        "generated_summary": state["summary"]
    })

    # 当图恢复执行时，interrupt() 的返回值 `result` 将会是稍后通过 Command(resume=...) 传入的数
    # 节点返回一个新字典，用人类编辑后的内容来更新状态中的 'summary' 字段。
    return {
        "summary": result["edited_summary"]
    }

# 节点C：模拟使用编辑后摘要的下游任务
def downstream_use(state: State) -> State:
    print(f"--- 步骤3: 正在使用编辑后的摘要... ---")
    # 打印最终确认的摘要，证明状态已经被人工输入所更新。
    print(f"✅ Using edited summary: {state['summary']}")
    return state

# 3. 构建图 (Graph)
builder = StateGraph(State)

# 将上面定义的函数注册为图中的节点
builder.add_node("generate_summary", generate_summary)
builder.add_node("human_review_edit", human_review_edit)
builder.add_node("downstream_use", downstream_use)
```

```
# 设置图的流程：定义入口和节点之间的固定连接顺序。
builder.set_entry_point("generate_summary")
builder.add_edge("generate_summary", "human_review_edit")
builder.add_edge("human_review_edit", "downstream_use")
builder.add_edge("downstream_use", END) # END 表示流程结束

# 4. 编译图
# 设置一个内存检查点（checkpointing）。这是使用 interrupt 功能的必要条件，
# 因为图需要一个地方来保存它暂停时的状态。
checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

# 5. 执行图 - 第一次调用（触发中断）
# 为本次运行创建一个唯一的线程ID（thread_id），用于追踪和恢复状态。
config = {"configurable": {"thread_id": str(uuid.uuid4())}}
print("--- 开始执行图 ---")
# 首次调用图。执行将会在 'human_review_edit' 节点中的 interrupt() 处暂停。
result = graph.invoke({}, config=config)

# 当图被中断时，返回的结果会包含一个 '__interrupt__' 键，其值是中断时发送的数据。
print("\n--- 图已暂停，等待输入 ---")
print("中断信息:", result["__interrupt__"])

# 6. 恢复图的执行 - 第二次调用（提供输入）
# 模拟人类用户提供了编辑后的摘要。
edited_summary = "The cat lay on the rug, gazing peacefully at the night sky."

print("\n--- 恢复执行图 ---")
# 使用 Command(resume=...) 来恢复图的执行。
# 传入的数据（一个字典）会成为 'human_review_edit' 节点中 interrupt() 函数的返回值。
# 注意：必须使用与第一次调用相同的 config（包含相同的 thread_id）来恢复正确的会话。
resumed_result = graph.invoke(
    Command(resume={"edited_summary": edited_summary}),
    config=config
)

print("\n--- 图执行完毕 ---")
# 打印图执行完毕后的最终状态，可以看到 'summary' 已经被更新。
print("最终状态:", resumed_result)
```

模式三：审查工具调用 (Review Tool Calls)

这是保障 Agent 安全性的终极防线。可以在 Agent 决定调用某个工具时强制中断，等待人工确认。下面会介绍如何在agent内部通过中断interrupt()实现人工干预。

环境安装



```
pip install -U langgraph
pip install -U langchain
pip install -U langchain_openai
```

步骤一：先用Venus AI平台初始化模型

```
BASE_URL=" http://v2.xxxxxx.com/llmproxy"
TOKEN=...
MODEL_NAME=... # 可以填写模型名称（e.g. gpt-4o, deepseek-v3.1-terminus,外部模型如gpt-4o, 需要
from langchain_openai import ChatOpenAI
from langchain.chat_models import init_chat_model

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN
    temperature=0,
)
```

步骤二：定义酒店查询工具

这里定义了一个酒店查询工具，后面会再引入MCP协议提供的智能搜索工具为例用LangGraph实现中断

```
def search_hotel(hotel_name: str):
    """根据酒店名称查询酒店信息"""
    return f"为您查询{hotel_name}."
```

注意：这个工具定义了一个参数hotel_name，定义了工具描述：根据酒店名称查询酒店信息，在步骤三和步骤四会被提到。

步骤三：定义可人工干预的通用的包装器：可为任何工具添加

通用包装器 add_human_in_the_loop：内部集成了interrupt()，这是个可以包装任何工具的函数。它能在不修改原工具代码的情况下，自动为其增加人工审查层，极大地提升了代码的模块化和复用性。

可以通过参数tool和interrupt_config来直接使用。

具体参见下面的代码和注释。

```
from typing import Callable
from langchain_core.tools import BaseTool, tool as create_tool
```

```python
from langchain_core.runnables import RunnableConfig
from langgraph.types import interrupt
from langgraph.prebuilt.interrupt import HumanInterruptConfig, HumanInterrupt

def add_human_in_the_loop(
    tool: Callable | BaseTool,
    *,
    interrupt_config: HumanInterruptConfig = None,
) -> BaseTool:
    """
    将一个工具封装起来，为其添加人工审核（Human-in-the-loop）功能。
    这是一个高阶函数，它接收一个工具，返回一个带有内置审批流程的新工具。
    参数:
        tool: 可以是一个普通的 Python 函数，也可以是一个继承自 BaseTool 的 LangChain 工具实例。
        interrupt_config: 一个可选的字典，用于配置人类审批界面的选项（如是否允许编辑、批准等）。
    返回:
        一个新的 BaseTool 实例，这个新工具封装了原始工具并加入了人工审核功能。
    """
    # 步骤 1: 规范化输入工具
    # 检查传入的 `tool` 是否已经是 `BaseTool` 的实例。
    # 如果不是（例如，只是一个普通的 Python 函数），则使用 `create_tool` 将其转换为一个标准的 Lang
    # 这确保了后续代码可以统一处理 `tool` 对象。
    ifnot isinstance(tool, BaseTool):
        tool = create_tool(tool)

    # 步骤 2: 设置默认的人工交互配置
    # 如果用户没有提供 `interrupt_config`，则设置一个默认配置。
    # 默认允许用户：批准 (accept)、编辑 (edit) 或直接回应 (respond)。
    if interrupt_config isNone:
        interrupt_config = {
            "allow_accept": True,
            "allow_edit": True,
            "allow_respond": True,
        }

    # 步骤 3: 创建并返回一个新的、封装后的工具
    # 使用 `create_tool` 装饰器来创建一个新的工具。
    # 关键在于，这个新工具会继承原始工具的名称 (`name`)、描述 (`description`) 和参数结构 (`args_s
    # 这使得调用此工具的 LLM 能够像使用原始工具一样看到它，而不知道其内部包含了人工审核逻辑。
    @create_tool(
        tool.name,
        description=tool.description,
        args_schema=tool.args_schema
    )
    def call_tool_with_interrupt(config: RunnableConfig, **tool_input):
        """这是新工具的具体实现函数，它包含了人工审核的逻辑。"""
```

```python
        # 步骤 3.1: 构建中断请求
        # 构建一个 `HumanInterrupt` 类型的字典，这个字典将作为 `interrupt` 函数的参数。
        # 它包含了所有需要展示给人类审批者的信息：
        # - action_request: AI 想要执行的动作（工具名和参数）。
        # - config: 告诉前端界面应该显示哪些按钮（批准、编辑等）。
        # - description: 给人类审批者的提示信息。
        request: HumanInterrupt = {
            "action_request": {
                "action": tool.name,
                "args": tool_input
            },
            "config": interrupt_config,
            "description": "Please review the tool call"
        }

        # 步骤 3.2: 暂停图并等待人类输入
        # 调用 `interrupt` 函数，将请求打包成列表传入。
        # **这是图执行暂停的地方**。
        # 当图通过 `resume` 命令恢复时，`interrupt` 函数会返回一个响应列表，我们取第一个响应。
        response = interrupt([request])[0]

        # 步骤 3.3: 根据人类的响应采取行动
        # 如果人类审批者点击了“批准”（`accept`）
        if response["type"] == "accept":
            # 直接使用原始的参数调用原始工具。
            tool_response = tool.invoke(tool_input, config)
        # 如果人类审批者编辑了参数（`edit`）
        elif response["type"] == "edit":
            # 从响应中获取新的参数，更新 `tool_input`。
            tool_input = response["args"]["args"]
            # 然后用新参数调用原始工具。
            tool_response = tool.invoke(tool_input, config)
        # 如果人类审批者选择直接回应（`response`），而不是执行工具
        elif response["type"] == "response":
            # 将其提供的反馈内容（`user_feedback`）直接作为工具的输出返回。
            # 这可以用来给 LLM 提供指导，例如：“不要搜索这个，请先总结一下已有信息。”
            user_feedback = response["args"]
            tool_response = user_feedback
        else:
            # 处理未知的响应类型，增加代码的健壮性。
            raise ValueError(f"Unsupported interrupt response type: {response['type']}")

        # 返回最终结果。这个结果将作为工具的执行输出，返回给调用它的 LLM。
        return tool_response

    # `add_human_in_the_loop` 函数返回这个被完全封装好的、带有新功能的新工具。
    return call_tool_with_interrupt
```

步骤四：定义可人工干预的预订酒店agent

使用**步骤三创建的包装器add_human_in_the_loop**和**步骤二定义的search_hotel工具**

search_tools

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.prebuilt import create_react_agent
# 1. 初始化检查点
# 同样，因为要使用中断功能，必须提供一个检查点来保存和恢复 Agent 的状态。
checkpointer = InMemorySaver()

# 2. 带人机交互的agent定义方法：
search_agent = create_react_agent(
    # 指定 Agent 使用的 LLM 模型
    model=model,
    # 为 Agent 提供工具列表
    tools=[
        # 关键点：我们没有直接传入 `search_tool`，
        # 而是传入了 `add_human_in_the_loop(search_tool)` 的返回结果。
        # 这意味着 Agent 拿到的 `search_hotel` 工具已经是被封装过的、带有人机交互逻辑的版本。
        # 上一节中定义的封装函数
        add_human_in_the_loop(search_hotel),
    ],
    # 将检查点与 Agent 关联起来
    checkpointer=checkpointer,
    name="search_assistant",
    prompt="你是一个酒店查询工具"
)
```

步骤五：配置并运行agent

Turn0:

输入query：我要查酒店，llm判断会用到步骤二定义的search_hotel工具

```
# 配置并运行 Agent
# 为本次对话创建一个唯一的线程 ID，以便状态可以被正确保存和恢复。
config = {"configurable": {"thread_id": "1"}}

# 使用 `stream` 方法来运行 Agent，这样我们可以观察到每一步的中间输出。
# 这对于理解 Agent 的思考过程和中断发生的位置非常有帮助。
# Run the agent
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "我要查酒店"}]},
```

```
    config
):
    print(chunk)
    print("\n")
# 注意：此代码运行后会暂停，等待下一次 `invoke` 或 `stream` 调用来恢复。
# 要完成整个流程，你需要像之前的例子一样，捕获中断信息，然后用 Command(resume=...) 再次调用 agent。
```

此时没有中断，而是要求输入酒店名称hotel_name,这个是因为我们在步骤二定义search_hotel这个工具的时候定义了hotel_name参数，只有当工具定义的参数信息都完备时才具体触发工具调用前的中断逻辑

运行结果：

```
/tmp/ipykernel_48599/2532366464.py:11: LangGraphDeprecatedSinceV10: create_react_agent has been moved to `langchain.agents`. Please upda
te your import to `from langchain.agents import create_agent`. Deprecated in LangGraph V1.0 to be removed in V2.0.
  agent = create_react_agent(
{'agent': {'messages': [AIMessage(content='好的。请说明您是要查询某一家具体酒店，还是需要我根据您的需求推荐酒店。\n\n如果是查询具体酒店：\n- 直接发送酒店名称，我会马上为您查询相关信息。\n\n如果需要我帮您推荐，请提供以下信息：\n- 城市或目的地\n- 入住和退房日期\n- 人数（成人/儿童）与房型偏好（大床/双床）\n- 预算范围（每晚）\n- 偏好（星级、位置如地铁/景点附近、是否含早、需要停车等）\n\n示例：上海｜11月10日入住-11月12日退房｜2位成人｜预算每晚600-800元｜靠近地铁，偏好四星。', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 620, 'prompt_tokens': 130, 'total_tok
ens': 750, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 448, 'rejected_predicti
on_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-2025
-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-CTTK3c0Am6KeNuaPtITn44Gk8U3qu', 'finish_reason': 'stop', 'logprobs': None}, id='lc_
run--88ece532-a96d-4c27-b9ca-65dbab07a97f-0', usage_metadata={'input_tokens': 130, 'output_tokens': 620, 'total_tokens': 750, 'input_tok
en_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 448}})]}}
```

Turn1:

输入query：帮我查下北京瑰丽酒店的信息

```
# 配置并运行 Agent
# 为本次对话创建一个唯一的线程 ID，以便状态可以被正确保存和恢复。
config = {"configurable": {"thread_id": "1"}}

# 使用 `stream` 方法来运行 Agent，这样我们可以观察到每一步的中间输出。
# 这对于理解 Agent 的思考过程和中断发生的位置非常有帮助。
# Run the agent
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "帮我查下北京瑰丽酒店的信息"}]},
    config
):
    print(chunk)
    print("\n")
# 注意：此代码运行后会暂停，等待下一次 `invoke` 或 `stream` 调用来恢复。
# 要完成整个流程，你需要像之前的例子一样，捕获中断信息，然后用 Command(resume=...) 再次调用 agent。
```

llm判断会用到步骤二定义的search_hotel工具，且识别出工具要求的参数hotel_name=北京瑰丽酒店，工具要求的信息已经完备，此时触发中断

运行结果：

```
{'agent': {'messages': [AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 94, 'prompt_tokens': 312, 'total_tokens': 406, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 64, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-CTTKiblEMedLwjHzONsxt0ZKNb30f', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--179a462f-5582-44e6-a7e5-d8d3bca2a983-0', tool_calls=[{'name': 'search_hotel', 'args': {'hotel_name': '北京瑰丽酒店'}, 'id': 'call_HLBLKBkUzMHYPVmpo6Kuwbz5', 'type': 'tool_call'}], usage_metadata={'input_tokens': 312, 'output_tokens': 94, 'total_tokens': 406, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 64}})]}}


{'__interrupt__': (Interrupt(value=[{'action_request': {'action': 'search_hotel', 'args': {'hotel_name': '北京瑰丽酒店'}}, 'config': {'allow_accept': True, 'allow_edit': True, 'allow_respond': True}, 'description': 'Please review the tool call'}], id='bdf50c43f27530cd4b9fd6f42e90c82f'),)}
```


步骤六：Command根据人工输入恢复代理以继续。

```python
from langgraph.types import Command

for chunk in agent.stream(
    Command(resume=[{"type": "accept"}]),
    config
):
    print(chunk)
    print("\n")
```

运行结果：


```
{'tools': {'messages': [ToolMessage(content='为您查询北京瑰丽酒店.', name='search_hotel', id='af48887c-c8b6-4dbb-ac51-d12006bde606', tool_call_id='call_BRh5CqfW6TVShLCkAufSgSMw')]}}


{'agent': {'messages': [AIMessage(content='我可以为您整理北京瑰丽酒店（Rosewood Beijing）的关键信息，并根据您的日期帮您查实时房价与空房。以下是概览，供您参考：\n\n- 酒店定位与位置：超豪华品牌瑰丽酒店集团在中国大陆的旗舰之一，位于朝阳区核心CBD，周边有国贸、央视大楼、世贸天阶、太古里三里屯等，商务与休闲都很方便。\n- 房型与特色：豪华客房、行政房与多种套房为主，房间普遍较大，设计偏居住化风格；部分房型配有步入式衣帽间、深浸浴缸、优质床品，服务细致。\n- 餐饮与酒吧：酒店内常见有中餐（主打北方菜与烤鸭）、火锅概念餐厅、法式/西式餐厅及露台酒吧等，品质口碑较好，适合商务会客与休闲聚会。\n- 康体设施：室内恒温泳池、24小时健身房、Sense Spa（水疗）与多功能会议/宴会空间。\n- 交通：临近多条地铁（常见为1号线/10号线的国贸、金台夕照、呼家楼一带），前往首都机场与大兴机场都较为便利；周边商务区高峰期车流较大。\n- 入住/退房时间：通常为15:00办理入住，12:00退房（实际以酒店当天政策为准）。\n- 亲子与加床：一般可提供婴儿床，部分房型可加床，需提前确认。\n- 宠物与吸烟：宠物及吸烟政策会随酒店具体规定而变动，建议提前核实；多数客房为无烟房。\n- 价格参考：标准客房淡季常见价位约人民币 2500–3500/晚，旺季或节假日可能在 3500–5000+；套房价格更高。具体视日期、活动与房态浮动。\n- 适合人群与口碑：服务与设计感受普遍好评，餐饮实力较强；价格较高、工作日CBD周边交通拥堵是常见反馈。\n\n如果您告诉我：\n- 计划入住的日期和晚数\n- 入住人数及是否需要加床/婴儿床\n- 预算范围与偏好的房型或景观\n我可以帮您查询最新的房价、可预订房型、餐厅营业时间与SPA预约情况，并提供可预订的官方链接或客服电话。也可以为您对比同区域的同档次酒店选项。', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 2418, 'prompt_tokens': 178, 'total_tokens': 2596, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 1792, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-CTTFfCWypDUSKdSKZzT4nN69Rd3HO', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--6cc8f121-73f6-4315-a262-b5300ee901d0-0', usage_metadata={'input_tokens': 178, 'output_tokens': 2418, 'total_tokens': 2596, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 1792}})]}}
```


模式四：验证用户输入 (Validate Human Input)

在需要用户提供特定格式的输入时，可以使用循环来反复请求，直到输入有效为止。

```python
# 导入所需库
from typing import TypedDict
import uuid

from langgraph.constants import START, END
from langgraph.graph import StateGraph
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver

# 1. 定义图的状态
# State 定义了在整个图的执行过程中共享的数据结构。
class State(TypedDict):
```

```python
    age: int

# 2. 定义图中的节点

# 节点A：获取并验证用户输入的年龄
# 这个节点展示了一个非常实用的模式：在节点内部循环，直到获得有效的人类输入。
def get_valid_age(state: State) -> State:
    # 初始化第一次向用户展示的提示信息
    prompt = "Please enter your age (must be a non-negative integer)."

    # 使用一个无限循环来不断请求输入，直到输入有效为止
    whileTrue:
        # 核心：调用 interrupt() 来暂停图的执行，并向用户显示提示信息。
        # 当图恢复时，`resume` 命令传入的值将成为 `user_input`。
        user_input = interrupt(prompt)

        # 验证用户的输入
        try:
            age = int(user_input)
            if age < 0:
                raise ValueError("Age must be non-negative.")
            break# 如果输入有效（是正整数），则跳出循环
        except (ValueError, TypeError):
            # 如果输入无效（例如，是字符串或负数），则更新提示信息，
            # 准备在下一次循环中向用户显示带有错误反馈的新提示。
            prompt = f"'{user_input}' is not valid. Please enter a non-negative integer f

    # 只有在获得有效输入并跳出循环后，此节点才会返回结果，更新图的状态。
    return {"age": age}

# 节点B：使用验证后的年龄
def report_age(state: State) -> State:
    # 打印最终确认的年龄，证明状态已成功更新。
    print(f"✅ Human is {state['age']} years old.")
    return state

# 3. 构建图
builder = StateGraph(State)
builder.add_node("get_valid_age", get_valid_age)
builder.add_node("report_age", report_age)

# 设置图的执行流程
builder.set_entry_point("get_valid_age")
builder.add_edge("get_valid_age", "report_age")
builder.add_edge("report_age", END)

# 4. 编译图
```

```
# 创建一个内存检查点，这是使用 interrupt 功能的必要条件。
# 它负责保存图在暂停时的完整状态（包括在哪个节点的哪一行）。
checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

# 5. 执行图 - 分步演示

# 为本次运行创建一个唯一的会话ID
config = {"configurable": {"thread_id": str(uuid.uuid4())}}

# 第一次调用：图启动并立即在 get_valid_age 节点内暂停
print("--- 第一次调用：请求输入 ---")
result = graph.invoke({}, config=config)
print("中断信息:", result["__interrupt__"])  # 输出: "Please enter your age..."

# 第二次调用：模拟用户输入一个无效的字符串
print("\n--- 第二次调用：模拟无效输入（字符串）---")
result = graph.invoke(Command(resume="not a number"), config=config)
print("中断信息:", result["__interrupt__"])  # 输出：带有验证失败信息的提示

# 第三次调用：模拟用户输入一个无效的负数
print("\n--- 第三次调用：模拟无效输入（负数）---")
result = graph.invoke(Command(resume="-10"), config=config)
print("中断信息:", result["__interrupt__"])  # 输出：带有验证失败信息的提示

# 第四次调用：提供有效的输入
print("\n--- 第四次调用：提供有效输入 ---")
final_result = graph.invoke(Command(resume="25"), config=config)
print("\n--- 图执行完毕 ---")
print("最终状态:", final_result)  # 应该包含有效的年龄并打印出报告
```

高级主题与注意事项

- 并行中断与批量恢复：如果 Graph 并行执行了多个包含中断的节点，可以通过 graph.get_state(config).interrupts 获取所有中断的列表，并构造一个映射 resume_map = {interrupt_id_1: value_1, ...}，通过 Command(resume=resume_map) 一次性恢复所有中断。

- 子图中的中断：当中断发生在子图中时，恢复操作会从父图中调用该子图的节点开头，以及子图中包含中断的节点开头同时重跑。

- 调试利器：静态中断：除了用于人机交互的动态中断，LangGraph 还支持在编译时设置静态中断（interrupt_before, interrupt_after）。这相当于在特定节点前后设置了断点，是单步调试和检查 Graph 状态的绝佳工具，但不推荐用于生产环境的人机交互流程。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

langGraph实战案例--引入MCP协议实现真实工具的中断

本示例将实现一个通过MCP协议提供的智能搜索工具的中断：该工具是一个综合网络搜索插件，支持通用网络搜索、酒店机票查询、学术检索、新闻咨询、图片搜索等功能，需要先在Venus MCP server市场 - 网络综合搜索工具创建实例，然后获取访问sse链接后使用。该工具均已实现MCP标准化接口，可直接被Agent调用。

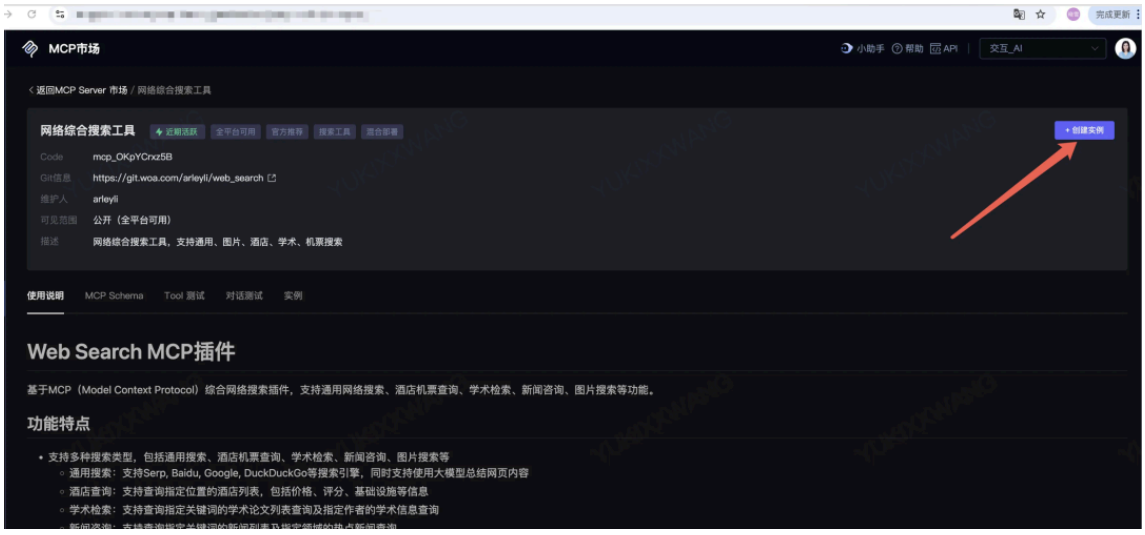
步骤一：先用Venus AI平台初始化模型

```
BASE_URL="  http://v2.xxxxxx.com/llmproxy"
TOKEN=...
MODEL_NAME=... # 可以填写模型名称（e.g. gpt-4o, deepseek-v3.1-terminus,外部模型如gpt-4o, 需要
from langchain_openai import ChatOpenAI
from langchain.chat_models import init_chat_model

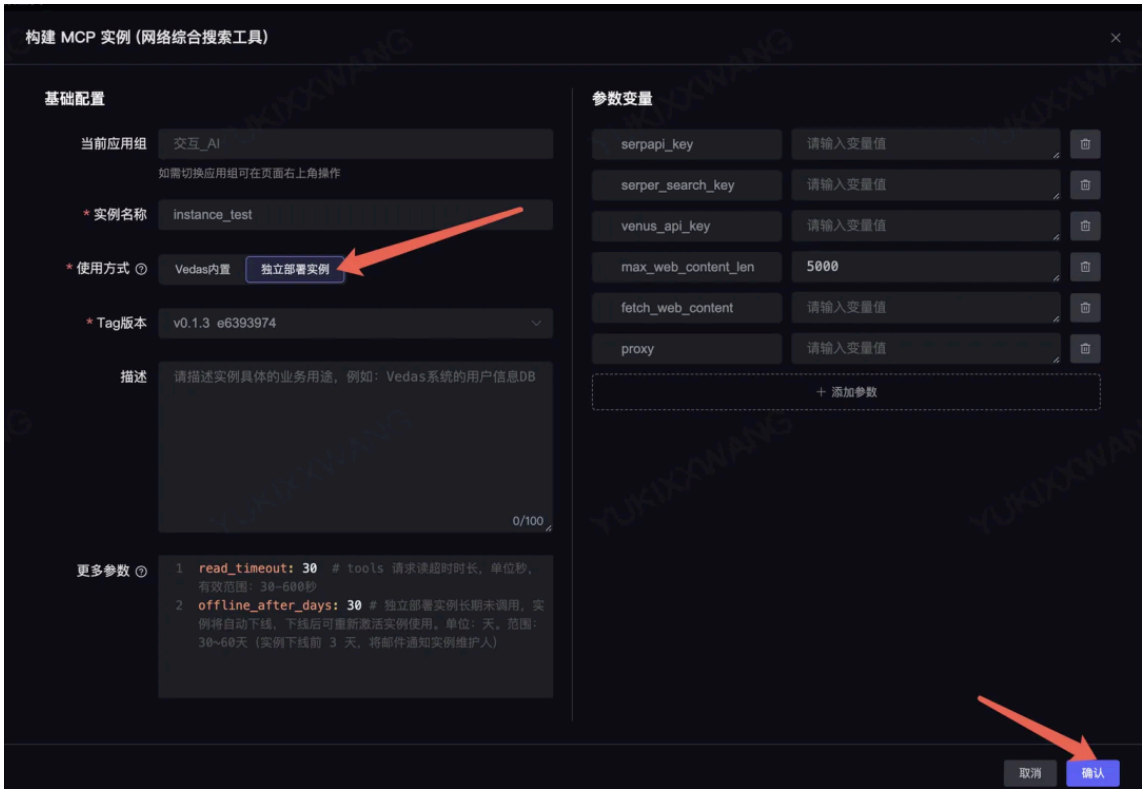
model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN
    temperature=0,
)
```

步骤二：在Venus MCP server市场 - 网络综合搜索工具创建实例，然后获取访问sse链接后使用

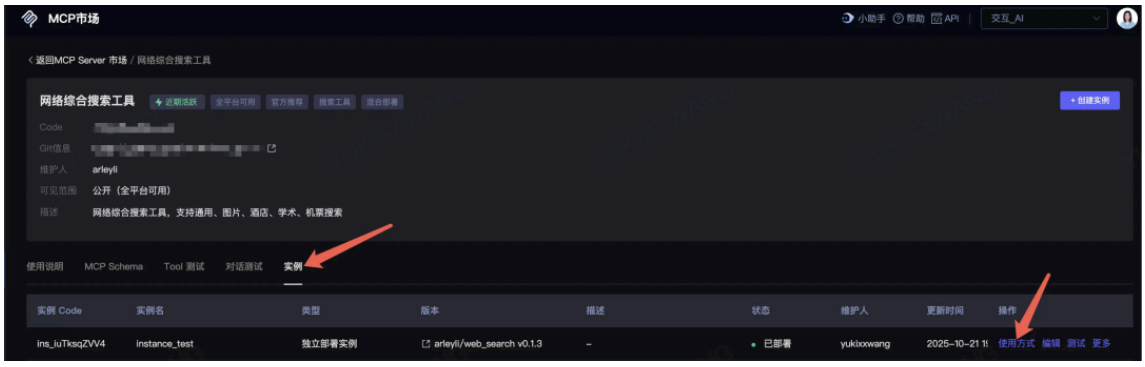
创建实例：



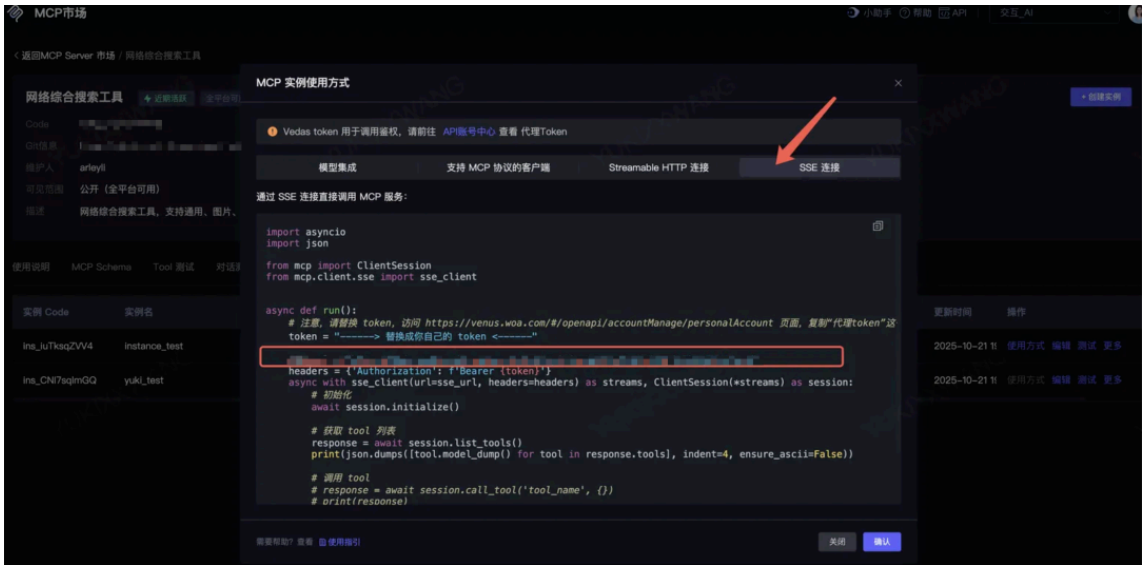
独立部署实例-->确认：



实例-->使用方式：



SSE连接--> sse_url



步骤三：引入venusMCPserver市场的智能搜多工具

以Venus MCP server市场 - 网络综合搜索工具为例用LangGraph实现中断

将步骤一的sse_url赋值给下面的url

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

search_client = MultiServerMCPClient(
    {
        "other_search": {
            "url": url, # 创建实例后填入实例生成的sse链接
            "headers": {
                "Authorization": f"Bearer {TOKEN}" # venus token，结尾有一个@XXX，XXX是应用组
            },
            "transport": "sse"}
      }
)
search_tools = await search_client.get_tools()
```

步骤四：定义可人工干预的通用的包装器：可为任何工具添加

由于要调用的工具是异步的，因此这个通用包装器实现过程中也要注意异步实现。与之前定义的 add_human_in_the_loop不同，有两处需要注意：

1. call_tool_with_interrupt函数的定义：

async def call_tool_with_interrupt(config: RunnableConfig, **tool_input): # <--- 1. 改为 async def

2. invoke改为ainvoke：

tool_response = await tool.ainvoke(tool_input, config) # <--- 2. 改为 await tool.ainvoke

完整实现见下面的代码：

```
# 异步
from typing import Callable
from langchain_core.tools import BaseTool, tool as create_tool
from langchain_core.runnables import RunnableConfig
from langgraph.types import interrupt
from langgraph.prebuilt.interrupt import HumanInterruptConfig, HumanInterrupt

def add_human_in_the_loop(
    tool: Callable | BaseTool,
    *,
    interrupt_config: HumanInterruptConfig = None,
```

```
) -> BaseTool:
    """
    将一个工具封装起来，为其添加人工审核（Human-in-the-loop）功能。
    这是一个高阶函数，它接收一个工具，返回一个带有内置审批流程的新工具。
    参数:
        tool: 可以是一个普通的 Python 函数，也可以是一个继承自 BaseTool 的 LangChain 工具实例。
        interrupt_config: 一个可选的字典，用于配置人类审批界面的选项（如是否允许编辑、批准等）。
    返回:
        一个新的 BaseTool 实例，这个新工具封装了原始工具并加入了人工审核功能。
    """
    # 步骤 1: 规范化输入工具
    ifnot isinstance(tool, BaseTool):
        tool = create_tool(tool)

    # 步骤 2: 设置默认的人工交互配置
    if interrupt_config isNone:
        interrupt_config = {
            "allow_accept": True,
            "allow_edit": True,
            "allow_respond": True,
        }

    # 步骤 3: 创建并返回一个新的、封装后的工具
    @create_tool(
        tool.name,
        description=tool.description,
        args_schema=tool.args_schema
    )
    asyncdef call_tool_with_interrupt(config: RunnableConfig, **tool_input):# <--- 1. 改为
        """这是新工具的具体实现函数，它包含了人工审核的逻辑。"""

        # 步骤 3.1: 构建中断请求
        request: HumanInterrupt = {
            "action_request": {
                "action": tool.name,
                "args": tool_input
            },
            "config": interrupt_config,
            "description": "Please review the tool call"
        }

        # 步骤 3.2: 暂停图并等待人类输入
        response = interrupt([request])[0]

        # 步骤 3.3: 根据人类的响应采取行动
        if response["type"] == "accept":
            # 直接使用原始的参数异步调用原始工具。
```

```
            tool_response = await tool.ainvoke(tool_input, config) # <--- 2. 改为 await to
        elif response["type"] == "edit":
            # 从响应中获取新的参数，更新 `tool_input`。
            tool_input = response["args"]["args"]
            # 然后用新参数异步调用原始工具。
            tool_response = await tool.ainvoke(tool_input, config) # <--- 2. 改为 await to
        elif response["type"] == "response":
            # 将其提供的反馈内容 (`user_feedback`) 直接作为工具的输出返回。
            user_feedback = response["args"]
            tool_response = user_feedback
        else:
            raise ValueError(f"Unsupported interrupt response type: {response['type']}")

        # 返回最终结果
        return tool_response

    return call_tool_with_interrupt
```

步骤五：定义可人工干预的搜索agent

使用**步骤四创建的包装器add_human_in_the_loop**和**步骤三定义的工具search_tools**，为 search_tools 中的每一个工具添加中断功能

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.prebuilt import create_react_agent
# 1. 初始化检查点
# 同样，因为要使用中断功能，必须提供一个检查点来保存和恢复 Agent 的状态。
checkpointer = InMemorySaver()


# 为 search_tools 中的每一个工具添加中断功能
wrapped_search_tools = [add_human_in_the_loop(tool) for tool in search_tools]

# 2. 带人机交互的agent定义方法：
agent = create_react_agent(
    # 指定 Agent 使用的 LLM 模型
    model=model,
    # 为 Agent 提供工具列表
    tools=wrapped_search_tools,
    # 将检查点与 Agent 关联起来
    checkpointer=checkpointer,
    name="search_assistant",
    prompt="你是一个能搜索各种信息的助手，支持通用网络搜索、酒店机票查询、学术检索、新闻咨询、图片搜索等
)
```

步骤六：配置并运行agent

输入query：查明天北京的天气

```
import asyncio

# 将调用 agent 的代码封装在一个 async 函数中
async def run_agent():
    config = {"configurable": {"thread_id": "2"}}

    # 使用 astream 进行异步迭代
    async for chunk in agent.astream(
        {"messages": [{"role": "user", "content": "查明天北京的天气"}]},
        config
    ):
        print(chunk)
        print("\n")
```

运行结果如下，触发中断：


```
{'agent': {'messages': [AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 228, 'prompt_tokens': 1958, 'total_tokens': 2186, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 192, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-CTWJKC8vMqHuXgLAiItnqqycgGjOT', 'finish_reason': 'tool_calls', 'logprobs': None}, name='search_assistant', id='lc_run--eaddb29f-e988-477e-84c4-5688b4d47082-0', tool_calls=[{'name': 'general_search', 'args': {'query': '北京 明天 天气 预报', 'num_results': 5}, 'id': 'call_Bi0feqtVgeyb6Q3lnEaSpFis', 'type': 'tool_call'}], usage_metadata={'input_tokens': 1958, 'output_tokens': 228, 'total_tokens': 2186, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 192}})]}}


{'__interrupt__': (Interrupt(value=[{'action_request': {'action': 'general_search', 'args': {'query': '北京 明天 天气 预报', 'num_results': 5}}, 'config': {'allow_accept': True, 'allow_edit': True, 'allow_respond': True}, 'description': 'Please review the tool call'}], id='a0bab4fd36a5a22e526c36618d036da5'),)}
```


步骤七：Command根据人工输入恢复代理以继续

```
import asyncio
from langgraph.types import Command

# 将调用 agent 的代码封装在一个 async 函数中
asyncdef run_response():
    config = {"configurable": {"thread_id": "2"}}



    print("--- Resuming with 'accept' ---")
    asyncfor chunk in agent.astream(
        Command(resume=[{"type": "accept"}]),
        config
    ):
        print(chunk)
        print("\n")

#run
await run_response()
```

人工同意之后，运行结果，输出检索到的天气情况：


```
--- Resuming with 'accept' ---
{'tools': {'messages': [ToolMessage(content='["{\\n  \\"url\\": \\"https://www.weather.com.cn/weather/101010100.shtml\\",\\n  \\"title\\": \\"北京天气预报,北京7天天气预报,北京15天天气预报,北京天气查询\\",\\n  \\"description\\": \\"22日（今天）. 小雨. 7℃ ; 23日（明天）. 多云. 13℃/3℃ ; 24日（后天）. 多云. 14℃/5℃ ; 25日（周六）. 多云转晴. 13℃/5℃ ; 26日（周日）. 晴. 13℃/3℃.\\"\\n}", "{\\n  \\"url\\": \\"https://weather.cma.cn/web/weather/\\",\\n  \\"title\\": \\"中国气象局-天气预报- 北京\\",\\n  \\"description\\": \\"11:00 · 10.5℃. 2.6m/s. 南风. 1032.8hPa ; 14:00 · 12.8℃. 3.3m/s. 西南风. 1029.2hPa ; 17:00 · 14.1℃. 2.9m/s. 南风. 1027.5hPa.\\"\\n}", "{\\n  \\"url\\": \\"https://www.nmc.cn/publish/forecast/ABJ/beijing.html\\",\\n  \\"title\\": \\"北京-天气预报 - 中央气象台\\",\\n  \\"description\\": \\"近日，今年下半年来最强冷空气的影响已逐渐展开，今天起将开始影响南方地区。预计未来几天，南方地区将陆续迎降温，而北方气温会持续偏低。\\"\\n}", "{\\n  \\"url\\": \\"https://www.accuweather.com/zh/cn/beijing/101924/weather-forecast/101924\\",\\n  \\"title\\": \\"北京, 北京市, 中國三日天氣預報 - AccuWeather\\",\\n  \\"description\\": \\"每日預報 ; 今天. 10/22. 57° 41°. 雲層不斷變化. 可能有陣雨 ; 周四. 10/23. 57° 37°. 多雲間晴. 大部晴朗 ; 周五. 10/24. 60° 41°. 晴有薄霧. 多雲.\\"\\n}"]', name='general_search', id='e0d11fde-9127-4fdd-893a-f034800fa53e', tool_call_id='call_Bi0feqtVgeyb6Q3lnEaSpFis')]}}


{'agent': {'messages': [AIMessage(content='明天（10月23日）北京天气：多云，最高气温约13°C，最低约3°C。昼夜温差大，早晚偏冷，外出建议加薄羽绒/厚外套。\n\n来源参考：中国天气网与中国气象局预报，实际情况可能随临近时段更新。需要我补充小时预报、降水概率、风向风力或空气质量吗？', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 616, 'prompt_tokens': 2646, 'total_tokens': 3262, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 512, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 1920}}, 'model_provider': 'openai', 'model_name': 'gpt-5-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-CTWJrIVbQUh5WkgBdc3L7uDZRsULs', 'finish_reason': 'stop', 'logprobs': None}, name='search_assistant', id='lc_run--643fc627-b62b-4060-be6b-b4f419593645-0', usage_metadata={'input_tokens': 2646, 'output_tokens': 616, 'total_tokens': 3262, 'input_token_details': {'audio': 0, 'cache_read': 1920}, 'output_token_details': {'audio': 0, 'reasoning': 512}})]}}
```


总结

LangGraph的“人工干预”为我们提供了一种非常强大且灵活的方式来将人类的智慧融入到AI的工作流程中。LangGraph通过interrupt机制，将“暂停-保存-恢复-重执行”的流程无缝集成到Agent 的工作流中，它为开发者提供了前所未有的控制力，使得我们能够构建出既能发挥大模型强大能力，又能在关键时刻接受人类智慧指导的复杂、可靠且安全的智能代理。通过运用持久化执行状态、动态及静态中断、以及灵活的集成点，我们可以实现各种各样的应用场景，比如审批与拒绝、编辑图状态、审查工具调用、验证人类输入等功能。这些模式可以帮助我们提高LLM应用的可靠性、安全性和准确性，让AI真正成为我们的好帮手。

参考文献

Overview



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容

公众号



阅读 6429

留言

写留言

还没有留言，来写首评